

From Local Whole-Body VLA Behaviors to Scene-Scale Aerial Manipulation

Anonymous Authors

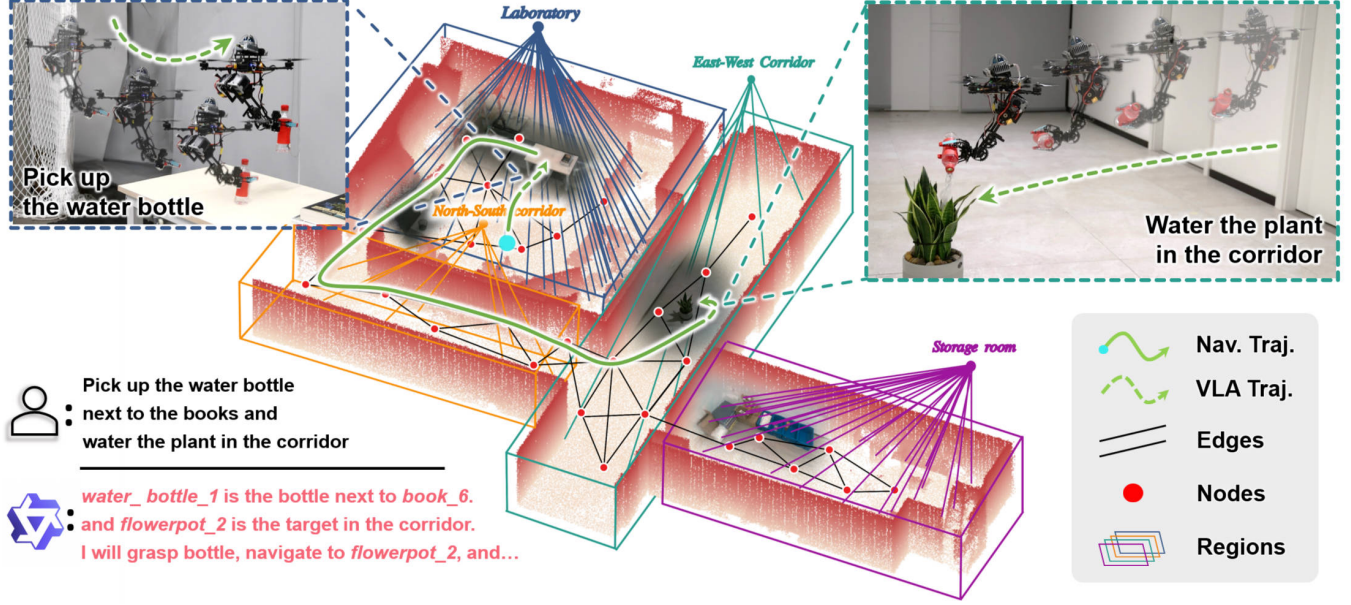


Fig. 1: The complete cross-site aerial manipulation process of picking up a water bottle and watering a plant. The language instruction is grounded to target object instances and feasible interaction regions through the relational Scene Graph. Solid and dashed green curves denote topology-guided navigation trajectories and local whole-body VLA trajectories, respectively. The left and right insets show the local VLA behaviors for bottle grasping and plant watering.

Abstract—Vision-language-action (VLA) models enable task-conditioned interaction, but extending them to scene-scale aerial manipulation remains challenging due to costly whole-body demonstrations, latency-induced action-state misalignment, and cross-site behavior composition. We present a unified framework for synthetic policy training and scene-scale execution on articulated uncrewed aerial manipulators (UAMs). A scene-reconfigurable pipeline synthesizes task-conditioned, kinodynamically feasible trajectories and synchronized multiview observations for VLA training without physical-platform demonstrations. Measured-progress-aligned realization (MPAR) aligns asynchronously returned action chunks with measured execution progress and realizes them as continuous, dynamically feasible trajectories. A relational Scene Graph grounds language goals to object instances and feasible interaction regions, while topology-guided transfer connects local behaviors across sites. Local VLA skills achieve 39/60 successes (65.0%) in simulation under oracle target and feasible-handoff conditions. Under 500-ms added latency, with and without a transient command-update stall, MPAR reduces median takeover phase error by 0.212 s over nominal-time alignment. The complete system completes 21/50 simulated multi-site missions (42.0%) and is further validated on a physical articulated UAM.

I. INTRODUCTION

Vision-language-action (VLA) models map visual observations, language instructions, and proprioception to robot

actions [1, 2], drawing on sensorimotor experience across tasks and embodiments [3]. Recent work has begun extending this paradigm to aerial manipulation [4, 5]. For an uncrewed aerial manipulator (UAM) [6]–[8], a VLA policy offers a way to coordinate the aerial base, arm, and gripper for language-specified interactions. Aerial mobility also makes it possible to perform these interactions at spatially separated sites without traversable ground connections. Realizing this capability requires extending local whole-body behaviors into a coordinated scene-scale mission (Fig. 1).

This extension presents three challenges. First, collecting diverse task-conditioned demonstrations on physical UAMs is costly and risky because base–arm–gripper motions must satisfy coupled dynamics and actuation limits. Second, inference and execution proceed asynchronously: when a new action chunk arrives, elapsed time may no longer indicate how far the UAM has actually progressed. The chunk must therefore be aligned with the physical execution state and incorporated without command discontinuities or violations of the platform’s motion limits. Third, a relational language goal must be resolved not only to the intended object instance but also to a feasible whole-body handoff state from which the local behavior can begin. Reaching such states across spatially separated interaction sites additionally

requires collision-free, kinodynamically feasible transfer.

To address these challenges, we propose a unified framework that learns local whole-body VLA behaviors and composes them into scene-scale aerial missions. Rather than assigning an entire mission to a single policy, we retain learned local coordination while explicitly handling physical realization and cross-site transfer. We evaluate the framework in simulation and through representative physical trials. Our main contributions are:

- A scene-reconfigurable supervision pipeline combining kinodynamically feasible whole-body expert planning with synchronized multiview rendering to generate task-conditioned training data for physical deployment without demonstrations collected on the target UAM.
- Measured-progress-aligned realization (MPAR), which projects estimated handoff states onto incoming VLA paths, anchors continuous takeover to active command states, and jointly optimizes path progress and trajectory timing under whole-body constraints.
- A scene-graph-based composition interface that resolves relational targets, derives feasible whole-body handoff states from skill-conditioned interaction regions, and guides cross-site transfer using polyhedral topology.

II. RELATED WORK

A. Vision-Language-Action Models for Aerial Manipulation

VLA research for aerial robots spans language-conditioned navigation [9], simulated aerial manipulation [4], and deployment on physical platforms [5]. Existing VLA and visuomotor policies acquire supervision through simulator teleoperation, generated navigation trajectories, Gaussian-splatting-based augmentation, and handheld cross-embodiment demonstrations [4, 5, 9, 10]. These approaches either retain dependence on human demonstrations, focus primarily on navigation, augment visual appearance without generating new whole-body interactions, or express actions in an end-effector-centric space. For articulated UAMs, this leaves an important gap: automatically generating task-conditioned supervision that is both whole-body and kinodynamically feasible, without collecting demonstrations on the platform.

Physical deployment introduces a distinct challenge: converting learned action predictions into reliable robot motion. AirVLA [5] deploys a pretrained VLA on a physical quadrotor equipped with a gripper, using RTC [11] for asynchronous execution and payload-aware guidance. Flying Hand and UMI-on-Air demonstrate learned visuomotor control on physical articulated UAMs, with the latter incorporating embodiment-aware controller guidance during policy sampling, although neither employs a language-conditioned foundation VLA [10, 12]. More general runtime methods address observation–execution mismatch and execution-time context prediction at the policy or action-sequence level [13, 14], while model-based aerial planners generate dynamically feasible trajectories from specified references or goals [15, 16]. These two lines address execution latency and

motion feasibility largely separately. For articulated UAMs, reliably realizing rolling whole-body VLA action chunks under variable inference latency requires both alignment with actual execution progress and continuous, constraint-aware trajectory realization.

B. Scene Graphs for Scene-Scale Aerial Manipulation

Open-vocabulary 3D scene representations organize object instances and their spatial relations for context-aware retrieval and large-scale navigation [17, 18]. Building on these representations, SayPlan and DovSG support long-horizon task planning and combined navigation–manipulation through structured scene reasoning [19, 20]. These systems primarily resolve language goals to semantic instances, regions, or high-level skills. For articulated UAMs, however, we additionally require grounding to an interaction region that admits a feasible whole-body state for initiating the local skill.

Beyond semantic grounding, hierarchical scene and topological graphs provide global connectivity. USS-Nav [21] couples polyhedral connectivity, semantic regions, and object instances for LLM-augmented UAV navigation, but primarily supports symbolic reasoning or point-level navigation. Safe-flight-corridor and whole-body planners, however, generate collision-free, dynamically feasible motion from specified geometric targets [15, 16], but do not resolve relational language goals. Scene-scale aerial manipulation therefore requires bridging relational grounding with executable interaction-region selection and whole-body-feasible transfer.

III. SYSTEM OVERVIEW

Given a natural-language mission ℓ and a preconstructed Scene Graph $\mathcal{G}$ registered to a metric map, we consider an articulated UAM comprising an aerial base, a multi-joint arm, and a gripper.

Within this setting, as shown in Fig. 2, the VLA is trained offline on synthesized whole-body demonstrations. At runtime, the mission layer queries $\mathcal{G}$ to resolve target instances and local instructions, while topology-guided transfer brings the UAM to a feasible handoff region. After a verified handoff, the VLA maps multiview images, proprioception, and the local instruction to finite-horizon base–arm–gripper action chunks. MPAR uses the measured state to align asynchronously returned chunks and generate continuous, constraint-aware trajectories. Visual and proprioceptive feedback close the local execution loop, while skill outcomes advance the mission.

IV. SCENE-RECONFIGURABLE WHOLE-BODY VLA SUPERVISION

To synthesize whole-body VLA supervision for physical-UAM deployment, we reconfigure each task at both the geometric and visual levels. For each episode, the pipeline samples the UAM initial state together with target and distractor identities and poses within task-valid ranges. It

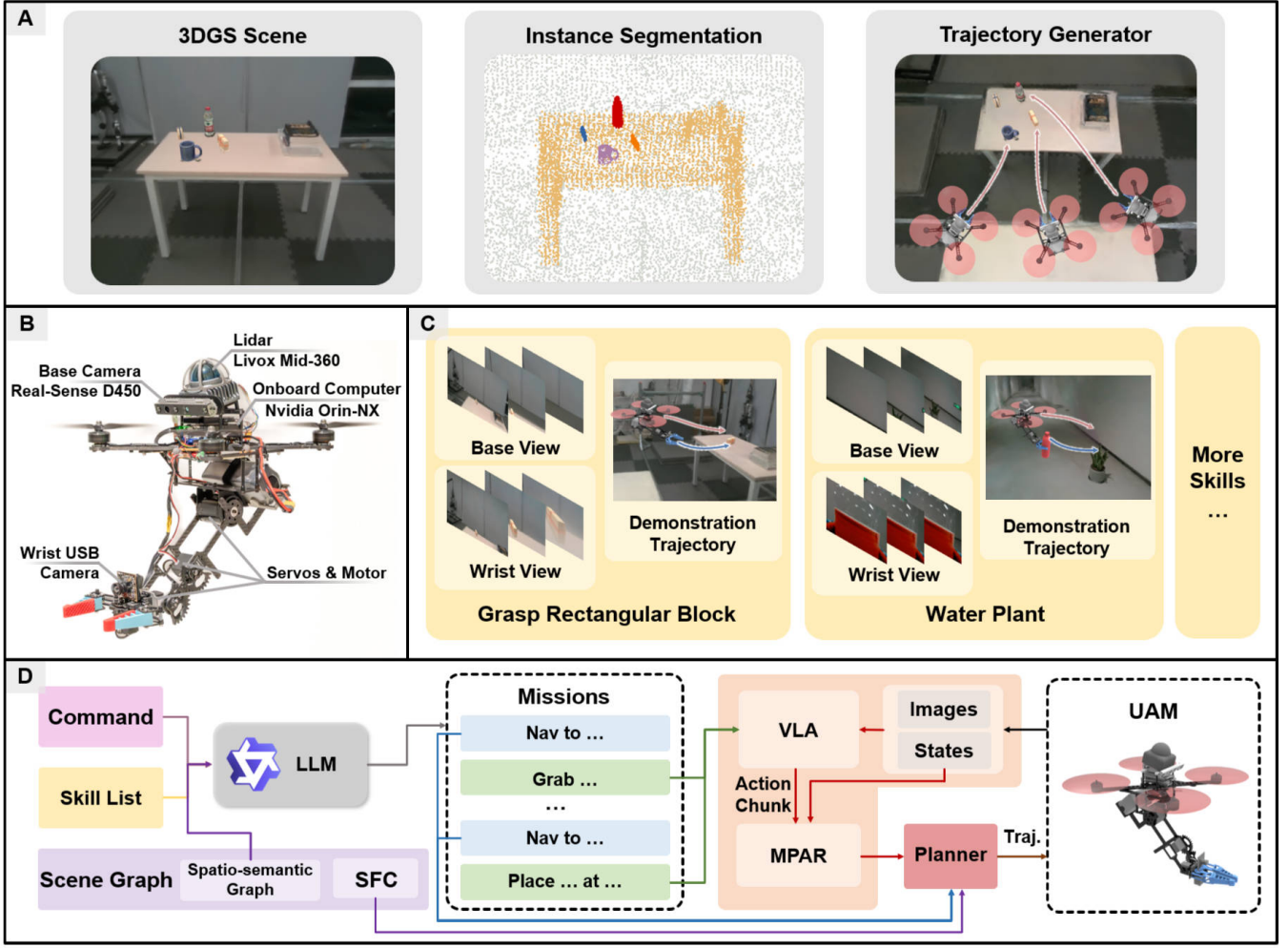


Fig. 2: System overview. (A) Reconfigurable 3DGS-based supervision synthesis. (B) Physical articulated UAM. (C) Synthesized local-skill observations and expert trajectories. (D) Online execution combining Scene Graph grounding and transfer, whole-body VLA prediction, MPAR, and kinodynamic trajectory realization.

then derives an object-relative interaction goal from the sampled configuration and language instruction. A whole-body planner generates coordinated aerial-base, arm, and gripper references under motion, joint, continuity, and collision constraints, retaining feasible rollouts as expert behaviors. Separate Gaussian representations of the static environment and movable assets are registered in a common metric frame, allowing assets to be independently placed with six-DoF poses while preserving the deployment-scene appearance. At each sampled time t , the pipeline renders synchronized base- and wrist-camera images from the expert state. These images, together with the whole-body state $\mathbf{x}_t$ and language instruction ℓ , form the policy input, while the corresponding expert future action chunk $A_{t:t+H}$ serves as the supervision target.

We instantiate the policy by fine-tuning a pretrained $\pi_{0.5}$ model through OpenPI [22]. Rank-16 and rank-32 LoRA adapters are applied to its PaliGemma language module and action expert, respectively. The model receives 224×224 base- and wrist-camera images, proprioception, and a language instruction, and predicts ten future actions at 0.25-s intervals (2.5-s horizon). At observation time t , the

proprioceptive state and the k -th action are

$$\mathbf{z}_t = [\boldsymbol{\theta}_t, \mathbf{v}_t^B, \phi_t, \vartheta_t] \in \mathbb{R}^8,$$

$$\mathbf{a}_{t,k} = [\boldsymbol{\theta}_{t,k}^*, g_{t,k}, \Delta \mathbf{p}_{t,k}^B, \Delta \psi_{t,k}] \in \mathbb{R}^8.$$

Here, $\boldsymbol{\theta}_t$ and $\boldsymbol{\theta}_{t,k}^*$ are the current and target 3-DoF arm configurations, $\mathbf{v}_t^B$ is the base translational velocity in the yaw-aligned body frame, and ϕ_t and ϑ_t are roll and pitch. The action additionally contains a gripper-closing score $g_{t,k}$ and base-position and yaw offsets relative to the observation-time base pose. All waypoints in a chunk share this anchor rather than being accumulated recursively, and $g_{t,k} > 0.5$ denotes closing. During fine-tuning, the joint targets are delta-encoded relative to $\boldsymbol{\theta}_t$ and converted back to absolute targets before runtime realization. Simulation and physical policies are separately fine-tuned on MuJoCo-collected and 3DGS-rendered data, respectively (39,200 windows each), with dataset-specific quantile normalization. AdamW is used in training for 30,000 updates with a batch size of 32 and cosine learning-rate decay from 2.5×10^{-5} to 2.5×10^{-6} after 1,000 warmup steps. Each policy uses its final step-29999 checkpoint, fixed a priori.

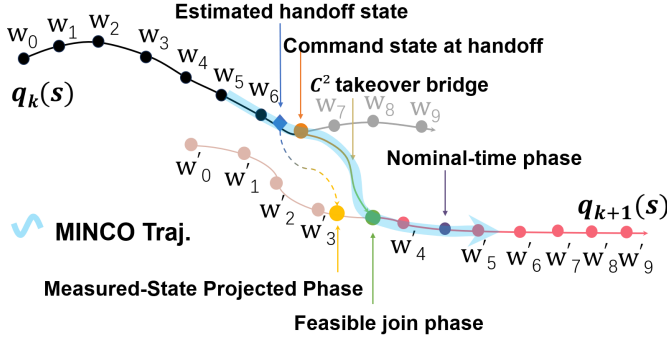


Fig. 3: Measured-progress-aligned takeover between consecutive VLA chunks. Nominal-time alignment selects a phase according to elapsed time, whereas MPAR projects the estimated handoff state onto the incoming VLA path. Starting from the active command state at handoff, a C^2 bridge reaches a forward feasible join phase, after which MINCO realizes the remaining VLA-path suffix as a continuous trajectory.

V. MEASURED-PROGRESS-ALIGNED WHOLE-BODY REALIZATION

A. Whole-Body VLA Path Representation

As illustrated in Fig. 3, at observation time t_o , the VLA predicts an action chunk $\mathcal{A}_o = \{\mathbf{a}_k\}_{k=0}^K$, which is decoded into synchronized whole-body waypoints $\mathbf{w}_k = (\mathbf{p}_k, \psi_k, \boldsymbol{\theta}_k, g_k, s_k)$. Here, $\mathbf{p}_k$, ψ_k , and $\boldsymbol{\theta}_k$ denote the aerial-base position, yaw, and arm configuration, respectively; g_k is the gripper command; and $s_k = (k+1)\Delta t$ is the nominal offset from t_o , such that $k=0$ denotes the first future waypoint rather than a state at the observation time. Piecewise interpolation of the continuous components defines the geometric whole-body path

$$\mathbf{q}(s) = [\mathbf{p}^\top(s) \quad \psi(s) \quad \boldsymbol{\theta}^\top(s)]^\top, \quad s \in [s_0, s_K]. \quad (1)$$

A shared phase s indexes the base, yaw, and arm jointly, thereby preserving their coordination within the VLA-predicted behavior. Although s_k inherits the nominal temporal spacing of the action chunk, s is subsequently treated as path progress rather than a prescribed execution clock; its timing is determined during physical realization. Discrete gripper commands retain their source-waypoint associations and are mapped to the managed path after execution starts, triggering with retimed geometric progress rather than the original chunk timestamps.

B. Measured-Progress Alignment and Continuous Takeover

When a new action chunk is received at t_r , the runtime schedules its handoff at $t_h = \max(t_o + T_{\text{target}}, t_r + T_{\text{lead}})$, where T_{lead} reserves time for planning and validation. The currently committed trajectory remains active during inference, and t_h is restricted to its valid horizon. Let $\mathbf{x}_m(t_r)$ and $\mathbf{x}_c^-(t_r)$ denote the measured and commanded whole-body states, respectively. Their residual is propagated over $\Delta t_h = t_h - t_r$ to estimate the state at handoff. For example,

$$\begin{aligned} \hat{\mathbf{p}}_h &= \mathbf{p}_c^-(t_h) + \Delta \mathbf{p} + \Delta \mathbf{v} \Delta t_h + \frac{1}{2} \Delta \mathbf{a} \Delta t_h^2, \\ \hat{\boldsymbol{\theta}}_h &= \boldsymbol{\theta}_c^-(t_h) + \Delta \boldsymbol{\theta} + \Delta \dot{\boldsymbol{\theta}} \Delta t_h, \end{aligned} \quad (2)$$

with analogous propagation for velocity and yaw.

The observation clock gives the nominal path phase $s_{\text{nom}} = t_h - t_o$. We instead obtain a measured-aligned phase by projecting the estimated handoff state onto the fresh whole-body path within a bounded neighborhood $\mathcal{W}(s_{\text{nom}})$:

$$\hat{s} = \arg \min_{s \in \mathcal{W}(s_{\text{nom}})} D(\hat{\mathbf{x}}_h, \mathbf{q}(s)), \quad (3)$$

where D measures normalized disagreement between the predicted handoff state and the path configuration together with its phase-derived motion at s . The correction is accepted only if it satisfies recovery bounds and improves agreement over s_{nom} ; otherwise, $\hat{s} \leftarrow s_{\text{nom}}$.

For each valid handoff, let $\tilde{s}_h$ denote the phase selected online, where $\tilde{s}_h = s_{\text{nom}}$ for nominal-time alignment and $\tilde{s}_h = \hat{s}$ for MPAR. We define

$$\begin{aligned} s_h^* &= \arg \min_{s \in [s_0, s_K]} D(\mathbf{x}_m(t_h), \mathbf{q}(s)), \\ \delta s &= \tilde{s}_h - s_h^*, \quad e_s = |\delta s|, \\ L_p(a, b) &= \int_{\min(a, b)}^{\max(a, b)} \|\mathbf{p}'(s)\|_2 ds, \\ \ell_{\text{skip}} &= \mathbb{I}[\delta s > 0] L_p(s_h^*, \tilde{s}_h), \\ \ell_{\text{repeat}} &= \mathbb{I}[\delta s < 0] L_p(s_h^*, \tilde{s}_h). \end{aligned} \quad (4)$$

Here, s_h^* is the post-hoc phase obtained by projecting the state measured at handoff onto the incoming VLA path. Since s inherits the nominal waypoint timing, e_s is measured in seconds; positive and negative δs indicate skipped and repeated aerial-base path portions, respectively. Episode-level arc lengths are summed over valid handoffs.

From $\hat{s}$, the runtime searches for a feasible join phase s_{join} and bridge duration T_b . The bridge uses the exact state of the active command at t_h ,

$$\mathbf{b}(0) = \mathbf{x}_c^-(t_h), \quad \mathbf{b}(T_b) = \mathbf{q}(s_{\text{join}}), \quad (5)$$

while matching the endpoint derivatives. Candidate transitions are rejected if they violate base velocity, acceleration, yaw-rate, or joint-rate limits. Importantly, $\hat{\mathbf{x}}_h$ is used only to identify the appropriate VLA-path suffix; retaining $\mathbf{x}_c^-(t_h)$ as the bridge boundary ensures a continuous command handoff. If no admissible takeover is found, the chunk is rejected while the committed trajectory remains active and replanning proceeds from a fresh observation.

C. Progress-Parameterized MINCO Realization

The bridge and remaining VLA-path suffix define the reference for constrained trajectory realization. Following the QuadHand whole-body planning framework, we parameterize the executable flat output

$$\mathbf{z}(t) = [\mathbf{p}^\top(t) \quad \psi(t) \quad \boldsymbol{\theta}^\top(t)]^\top \quad (6)$$

using $\mathfrak{T}_{\text{MINCO}}^4$ for the aerial-base position and $\mathfrak{T}_{\text{MINCO}}^2$ for yaw and each arm joint [15]. The initial boundary is fixed to the active command state at t_h , preserving position, velocity, and acceleration for the base and state and rate for yaw and the arm. Since the bridge serves only to establish this continuous takeover, VLA-path tracking begins after s_{join} .

For the remaining M trajectory intervals, we jointly optimize their durations T_i and vertex phases s_i , using $\eta \in \mathbb{R}^{M-1}$ and $\eta_\rho \in \mathbb{R}$ to parameterize monotonic progress:

$$s_i = s_{\text{join}} + (s_K - s_{\text{join}}) \rho \sum_{j=1}^i \pi_j, \quad (7)$$

$$\pi = \text{softmax}([\eta^\top, 0]^\top), \quad \rho = \text{sigmoid}(\eta_\rho).$$

which guarantees $s_{\text{join}} < s_1 < \dots < s_M < s_K$. Together with the optimized durations, these phases determine the execution rate $\dot{s} \approx (s_i - s_{i-1})/T_i$, allowing the UAM to accelerate or slow its traversal without changing the ordering of the VLA behavior.

Denoting the MINCO intermediate waypoints by $\mathcal{P}$, the resulting optimization is written compactly as

$$\min_{\mathcal{P}, \mathbf{T}, \eta, \eta_\rho} J_{\text{smooth}} + J_{\text{cont}} + J_{\text{prog}} + J_{\text{feas}}, \quad (8)$$

where J_{smooth} penalizes the high-order derivatives of the MINCO trajectories. J_{cont} penalizes cross-track deviation more strongly than along-path error and tracks the path-indexed yaw and arm configurations, allowing the optimized progress $s(t)$ to absorb timing variations while preserving the geometric intent of the VLA-predicted behavior. J_{prog} regularizes phase distortion and progress-rate variation and discourages stagnation. J_{feas} penalizes violations of platform velocity, acceleration, body-rate, yaw-rate, and joint-rate limits, together with whole-body collisions. A densely sampled feasibility check is applied before commitment.

VI. SCENE-GRAPH-GUIDED SCENE-SCALE BEHAVIOR COMPOSITION

A. Scene-Graph-Conditioned Mission Composition

Building on the polyhedral spatial connectivity and semantic-region hierarchy of USS-Nav [21], our Scene Graph organizes object instances within an Area-Polyhedron hierarchy and encodes their spatial relations and free-space connectivity. Given a natural-language mission ℓ , an externally hosted LLM uses this graph context and the available local VLA skills to map linguistic references to graph-supported object categories and relations, resolve implicit task dependencies, and compose an ordered graph-grounded mission plan:

$$\Pi_\ell = (u_1, \dots, u_N), \quad (9)$$

$$u_i \in \{\text{NAVIGATE}, \text{EXECUTEVLA}, \text{RECOVER}\}.$$

Each step refers to a graph-described task target rather than specifying metric coordinates or control commands. The sequence defines mission logic and is instantiated by a deterministic executor as a state machine, whose transitions are advanced according to navigation and VLA outcomes.

B. Instance Grounding and Interaction-Region Construction

Each target-dependent action u_i carries a graph-supported relational description $\mathcal{Q}_i = (c_i, \Phi_i)$. A deterministic resolver selects object nodes matching the category and all specified relations, and execution proceeds only when the

Algorithm 1 Scene-Graph-Grounded Mission Execution

Require: Mission ℓ , Scene Graph $\mathcal{G}$, VLA skills $\mathcal{K}$
Ensure: Mission outcome y

- 1: $\Pi_\ell \leftarrow \text{LLMCOMPOSE}(\ell, \mathcal{G}, \mathcal{K})$
- 2: $\mathcal{S} \leftarrow \text{INSTANTIATEFSM}(\Pi_\ell)$
- 3: **while** $\mathcal{S}$ is not terminal **do**
- 4: $(\mathcal{Q}_i, \kappa_i) \leftarrow \text{CURRENTSTEP}(\mathcal{S})$
- 5: $(v_i^*, \mathcal{H}_i^*) \leftarrow \text{GROUNDANDFILTER}(\mathcal{Q}_i, \kappa_i, \mathcal{G})$
- 6: $(\mathbf{h}_i^*, \Gamma_i) \leftarrow \text{TOPOLOGYSEARCH}(\mathbf{x}, \mathcal{H}_i^*, \mathcal{G})$
- 7: **if** $v_i^* = \emptyset$ or $\mathcal{H}_i^* = \emptyset$ or $\Gamma_i = \emptyset$ **then**
- 8: $\mathcal{S} \leftarrow \text{UPDATEFSM}(\mathcal{S}, \text{REJECT})$
- 9: **continue**
- 10: **end if**
- 11: $\mathcal{F}_i \leftarrow \text{ASSEMBLECORRIDOR}(\Gamma_i, \mathcal{G})$
- 12: $r_i^{\text{nav}} \leftarrow \text{EXECUTETRANSFER}(\mathcal{F}_i, \mathbf{h}_i^*)$
- 13: **if** r_i^{nav} and $\text{VERIFYHANDOFF}(\mathbf{x}, \mathbf{h}_i^*)$ **then**
- 14: $r_i \leftarrow \text{EXECUTEVLA}(\kappa_i, v_i^*, \text{MPAR})$
- 15: **else**
- 16: $r_i \leftarrow \text{TRANSFERFAILURE}$
- 17: **end if**
- 18: $\mathcal{S} \leftarrow \text{UPDATEFSM}(\mathcal{S}, r_i)$
- 19: **end while**
- 20: **return** $y \leftarrow \text{OUTCOME}(\mathcal{S})$

query identifies a unique instance, i.e., $|\mathcal{C}(\mathcal{Q}_i)| = 1$. For the resolved instance v_i^* , a skill-conditioned geometric template defines an interaction region $\mathcal{I}_{\kappa_i}(v_i^*)$ rather than a single handoff pose. Intersecting this region with clearance-eroded reachable Polyhedra and enforcing whole-body feasibility yields the candidate handoff set $\mathcal{H}_i^*$ for subsequent transfer and VLA execution.

C. Topology-Guided Whole-Body Transfer and VLA Handoff

Each candidate handoff state $\mathbf{h} \in \mathcal{H}_i^*$ is mapped to its containing Polyhedron. From the current cell, graph search jointly selects a terminal handoff state $\mathbf{h}_i^*$ and a connected Polyhedron route $\Gamma_i = (P_0, \dots, P_L)$ according to route cost and terminal feasibility. The selected Polyhedron sequence provides the global route structure, while its projected cell regions and inter-cell connections define an aerial corridor that constrains whole-body transfer between interaction sites. Upon arrival, control is handed to the local VLA only when the measured pose and motion satisfy the interaction-region tolerances; otherwise, the handoff is rejected.

VII. EXPERIMENTS

A. Experimental Setup

We evaluate the proposed system in MuJoCo simulation and on a physical articulated UAM. The physical platform comprises a quadrotor, a 3-DoF arm, a gripper, an NVIDIA Jetson Orin NX, an Intel RealSense D450 depth camera, and a monocular RGB camera, while VLA inference runs remotely on an NVIDIA RTX 5090. Paired evaluations use the same model, initial conditions, task configurations, and random seeds. We report binary outcomes as counts, rates, and 95% Wilson confidence intervals, and continuous quantities by their median and 95th percentile (P95). Collision denotes unintended UAM contact, excluding task-prescribed gripper-object contact.

B. Cross-Site Mission Capability

TABLE I: Cross-site mission performance in simulation. Values denote successful trials over total trials.

Task	Whole-Task VLA	Ours
Water plant	0/10	5/10
Water and discard	0/10	3/10
Discard empty bottle	0/10	6/10
Store rectangular block	0/10	4/10
Store fan-shaped block	0/10	3/10
Overall	0/50	21/50
Success rate	0.0%	42.0%
95% Wilson CI	[0.0, 7.1]%	[29.4, 55.8]%

We evaluate cross-site mission execution in simulation by comparing our system with a Monolithic Whole-Task VLA under matched tasks, initial conditions, and random seeds. The baseline receives the complete mission instruction and generates rolling whole-body action chunks without relational grounding, topology-guided transfer, or feasible handoff. For a controlled comparison, its outputs are realized using the same MPAR module as ours. It is trained on 1,500 complete-task trajectories comprising 231,989 action windows, $5.92 \times$ the 39,200 MuJoCo local-skill windows used by our system. The baseline uses the same initialization, LoRA recipe, input-output representation, optimization budget, and fixed-checkpoint protocol (Sec. IV), with normalization computed from its own dataset. This comparison evaluates monolithic versus compositional scene-scale execution rather than isolating policy architecture or data efficiency.

Mission success criteria. A cross-site episode succeeds only when an independent evaluator observes every task-specific physical milestone in the prescribed order: object acquisition followed by retained watering; acquisition, retained watering, and release into the trash bin; acquisition and release into the trash bin; or block acquisition and release into the storage box. Acquisition, watering, and storage use the same physical predicates defined in Sec. VII-C; trash-bin placement analogously requires explicit release followed by stable residence in its prescribed target region. A collision, payload loss, missing or misplaced release, timeout, or runtime/planning termination constitutes episode failure, and component-reported completion alone is insufficient.

As shown in Table I, the monolithic baseline completes none of the 50 trials, whereas ours completes 21/50. Its failures comprise 19 collisions and 31 runtime-planning or action-feasibility terminations. Among our 29 failed trials, one results from planning failure and 28 occur during local VLA interaction. Under oracle target and feasible-handoff conditions, the local skills succeed in 39/60 interactions, comprising 14/30 object-acquisition and 25/30 post-acquisition trials, which further identifies object acquisition as the dominant local-behavior bottleneck.

C. Measured-Progress Whole-Body Realization

We compare our two constrained-realization variants against RTC-Direct, a direct-execution baseline based on

TABLE II: Task- and condition-wise success under runtime disturbances. Both views summarize the same 120 trials per method (40 per task and 30 per condition); best results are bold. Nominal-time and MPAR denote our ablation and full method, respectively.

Task/condition	RTC-Direct [5]	Nominal-time	MPAR
<i>Task-wise success</i>			
Grasp Rect	11/40	14/40	12/40
Watering	24/40	27/40	32/40
Storage	19/40	30/40	34/40
<i>Condition-wise success</i>			
C0	18/30	21/30	19/30
C1	10/30	17/30	20/30
C2	14/30	17/30	16/30
C3	12/30	16/30	23/30

the RTC mechanism used by AirVLA [5, 11]. RTC-Direct forwards RTC-conditioned action references without constrained trajectory realization. Our nominal-time ablation constructs continuous, constraint-aware MINCO trajectories but selects takeover locations according to nominal elapsed time. MPAR additionally aligns each returned chunk with measured physical progress. Because the two realization variants share the same trajectory formulation and constraints, their comparison isolates the contribution of measured-progress alignment. Unsafe publication denotes committing a MINCO trajectory that fails the feasibility check in Sec. V-C; it is inapplicable to RTC-Direct.

We evaluate all methods on rectangular-block grasping, plant watering, and storage placement under four controlled asynchronous-execution conditions, denoted C0–C3. C0 adds no artificial delay beyond native inference and communication; C1 and C2 delay each completed action chunk by an additional 250 and 500 ms, respectively. C3 augments C2 with one 250-ms hold of the latest base and arm commands per episode, emulating a transient command-update stall while physics and state estimation continue normally. These conditions progressively decouple elapsed time from physical execution progress. We conduct ten paired trials per task, method, and condition, yielding 360 trials.

Success criteria. Task success is determined from measured simulator states rather than workflow completion alone. Grasp Rect succeeds when, after gripper closure, the correct block is lifted clear of the tabletop while simultaneously contacting both finger pads with a normal force of at least 0.20 N on each side. Watering succeeds when the bottle remains grasped and the measured end effector stays within 0.22 m of the flower-pot center while satisfying $\theta_3 \geq \pi/2 - 0.03$ rad continuously for at least 3.0 s. Together with the calibrated tool-bottle geometry, this spatial condition keeps the bottle mouth above the pot during pouring. Storage succeeds when the correct block is explicitly released and, at least 0.35 s after opening the gripper, its center satisfies $|x - x_{\text{box}}| \leq 0.20$ m and $|y - y_{\text{box}}| \leq 0.22$ m, with object speed no greater than 0.15 m/s.

As shown in Table II, MPAR succeeds in 78/120 trials (65.0%), compared with 71/120 (59.2%) for our nominal-time ablation and 54/120 (45.0%) for RTC-Direct. Under

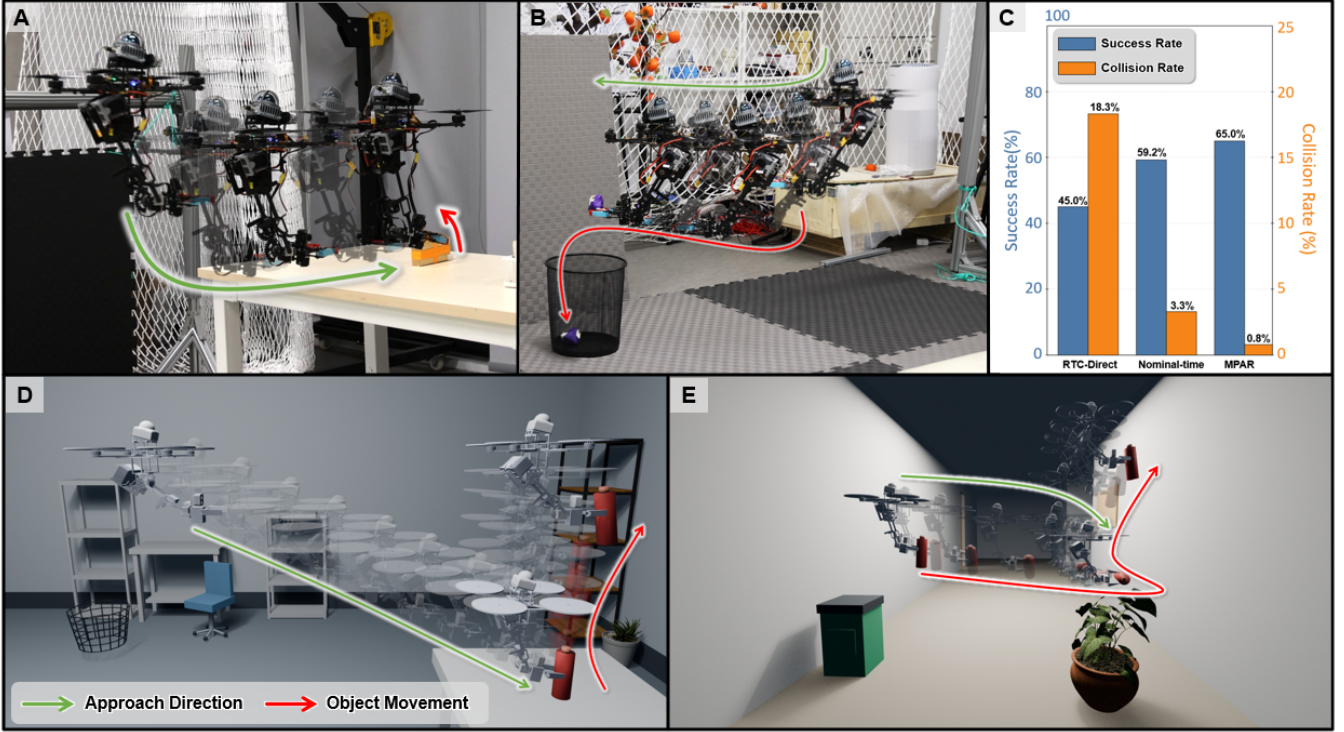


Fig. 4: **Evaluation of whole-body realization.** (A–B) Representative physical local interactions under MPAR. (C) Aggregate success and collision rates for RTC-Direct, the nominal-time ablation, and MPAR. (D–E) Representative simulated storage and watering executions. Green arrows indicate the UAM approach direction, red arrows indicate object motion, and ghosted configurations show the executed whole-body trajectory.

the predesignated C2–C3 conditions, the corresponding rates are 65.0%, 55.0%, and 43.3%. The largest separation occurs under C3, where MPAR reaches 23/30 successes, versus 16/30 and 12/30, respectively. The task-level effect is not universal: MPAR improves Watering and Storage but does not outperform the nominal-time ablation on Grasp Rect.

TABLE III: Mechanism-level descriptive comparison of nominal-time alignment and MPAR. Entries report task-balanced C2/C3 aggregates defined in the text; rejection is pooled across C2–C3. Lower is better.

Metric	Nominal-time	MPAR	Reduction
Phase P50 [s]	0.510/0.459	0.310/0.273	39.3/40.5%
Phase P95 [s]	0.830/0.831	0.444/0.508	46.6/38.9%
Skipped arc [cm]	8.8/8.7	3.7/3.1	58.0/64.4%
Repeated arc [cm]	12.1/10.3	7.8/5.2	35.5/49.5%
Rejection [%]	17.42	12.81	26.5%

Metrics are aggregated episode-first and task-balanced: phase-error P50/P95 are computed over valid handoffs within each episode, while skipped and repeated arc lengths are summed; each metric is then summarized by the median over ten episodes per task and the median across the three tasks.

The mechanism measurements in Table III further characterize the effect of progress alignment. Across 60 paired C2–C3 episodes, MPAR reduces episode-level P50 and P95 phase errors by an average of 0.212 and 0.315 s; their paired 95% confidence intervals, $[0.163, 0.262]$ and $[0.234, 0.393]$ s, exclude zero. It also reduces episode-level skipped and repeated path lengths by 7.44 and 1.89 cm,

respectively, while decreasing candidate-trajectory rejection from 17.42% to 12.81%. These gains are not accompanied by additional safety violations: collision-bearing runs decrease from 22/120 for RTC-Direct and 4/120 for our nominal-time ablation to 1/120 for MPAR, with no unsafe publications for either constrained variant. Representative physical and simulated executions are shown in Fig. 4.

D. Causal Ablation of the Scene-Graph Physical Interface

TABLE IV: Scene Graph physical-interface ablation. –G, –R, and –H remove relational grounding, topology-guided routing, and feasible handoff, respectively. Store, Water, and W→D (Water then Discard) are task-wise mission successes; Mission is their aggregate.

Metric	–G	–R	–H	Full
<i>Stage-level success</i>				
Grounding	20/35	35/35	35/35	35/35
Transfer	N/A	27/34	35/35	34/35
Handoff	N/A	26/27	26/35	34/34
Local	N/A	16/26	15/26	26/34
<i>Episode-level outcomes</i>				
Interface	0/15	8/15	7/15	14/15
Store	0/5	1/5	0/5	2/5
Water	0/5	2/5	1/5	4/5
W→D	0/5	1/5	1/5	3/5
Mission	0/15	4/15	2/15	9/15
Nav. collision	N/A	4/15	1/15	1/15

Notes: Local is event-level and may include multiple failures per episode; N/A denotes a stage not entered.

We evaluate the Scene Graph physical interface across

three cross-site mission types, with five paired episodes per task and variant. We compare the system with three single-component ablations: removing relational grounding, removing topology-guided routing, and replacing feasible-region handoff with a nominal point target. All variants use identical mission instructions, scene configurations, local VLA policies, MPAR realization, and fixed LLM-generated task sequences. Grounding is evaluated over all target references; other stage metrics are conditioned on entry. Each local event is one VLA skill invocation attempted once; after failure, later events proceed only if execution remains safe, without external reinitialization. For $-G$, all queries are audited before motion, and downstream stages are entered only if every query resolves the correct unique instance. Interface success, mission success, and navigation collision are episode-level; mission success additionally requires every prescribed local event.

As shown in Table IV, the complete system achieves interface success in 14/15 episodes and mission success in 9/15. Without relational grounding, no episode resolves all targets; removing routing increases navigation collisions, while nominal-point handoffs reduce handoff success. The task-wise breakdown covers sources of difficulty: Store Rect is dominated by object acquisition, whereas Water then Discard requires a longer three-interaction sequence.

E. Physical Full-Stack Validation

Under oracle target and feasible-handoff conditions, the local VLA completes bottle grasping, rectangular-block grasping, watering, and storage placement in 2/5, 1/5, 3/5, and 3/5 trials, respectively. These results demonstrate physical executability without demonstrations collected on the target UAM.

The full system also succeeds in 2/5 physical trials of the cross-site Water Plant mission.

VIII. CONCLUSION AND FUTURE WORK

This work unifies synthetic whole-body VLA training, measured-progress-aligned realization, and scene-graph-guided composition for scene-scale aerial manipulation, with simulation and physical validation. The current system relies on a preconstructed Scene Graph registered to a metric map and remote VLA inference, while its mission-level performance remains limited by individual local skills. Future work will investigate online Scene Graph construction and update, onboard inference, closed-loop failure processing, and broader physical-interaction capabilities.

REFERENCES

- [1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proceedings of the Conference on Robot Learning*, vol. 229, 2023, pp. 2165–2183.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, *et al.*, “OpenVLA: An open-source vision-language-action model,” in *Proceedings of the Conference on Robot Learning*, vol. 270, 2025, pp. 2679–2713.
- [3] Open X-Embodiment Collaboration, “Open X-Embodiment: Robotic learning datasets and RT-X models,” in *IEEE International Conference on Robotics and Automation*, 2024.
- [4] J. Sun, B. Tian, Q. Zhang, C. Li, Z. Song, Z. Cui, Y. Lv, and Y. Tian, “AIR-VLA: Vision-language-action systems for aerial manipulation,” *arXiv preprint arXiv:2601.21602*, 2026.
- [5] J. Tucker, D. Liu, A. Swann, A. Ren, J. Yu, J. Sun, B. Kim, L. McGranahan, Q. Vuong, and M. Schwager, “ π , but make it fly: Physics-guided transfer of VLA models to aerial manipulation,” *arXiv preprint arXiv:2603.25038*, 2026.
- [6] J. Lin, S. Ji, Y. Wu, T. Wu, Z. Han, and F. Gao, “Float drone: A fully-actuated coaxial aerial robot for close-proximity operations,” in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 7216–7223.
- [7] Y. Wang, R. Jin, X. Xu, H. Jin, R. Liu, Y. Yang, and L. Xie, “Ductam: A duct-assisted quadrotor-based aerial manipulator enabling high-force push-and-pull interactions,” 2026.
- [8] J. Li, Y. Chen, J. Kübel, H. Liu, J. Sugihara, and M. Zhao, “Effector-centric nmpc of tilttable-multirotors for offset-free omnidirectional aerial manipulation,” 2026.
- [9] X. Sun, W. Si, W. Ni, Y. Li, D. Wu, F. Xie, R. Guan, H.-Y. Xu, H. Ding, Y. Wu, Y. Yue, Y. Huang, and H. Xiong, “AutoFly: Vision-language-action model for UAV autonomous navigation in the wild,” *arXiv preprint arXiv:2602.09657*, 2026.
- [10] H. Gupta, X. Guo, H. Ha, C. Pan, M. Cao, D. Lee, S. Scherer, S. Song, and G. Shi, “UMI-on-Air: Embodiment-aware guidance for embodiment-agnostic visuomotor policies,” *arXiv preprint arXiv:2510.02614*, 2025.
- [11] K. Black, M. Y. Galliker, and S. Levine, “Real-time execution of action chunking flow policies,” in *Advances in Neural Information Processing Systems*, 2025.
- [12] G. He, X. Guo, L. Tang, Y. Zhang, M. Mousaei, J. Xu, J. Geng, S. Scherer, and G. Shi, “Flying hand: End-effector-centric framework for versatile aerial manipulation teleoperation and policy learning,” *arXiv preprint arXiv:2504.10334*, 2025.
- [13] H. Wang, G. Zhang, Y. Yan, Y. Shang, R. R. Kompella, and G. Liu, “Real-time robot execution with masked action chunking,” in *International Conference on Learning Representations*, 2026.
- [14] H. Jiang, Y. Zou, B. Liang, B. Liu, F. Meng, and S. Liu, “FutureRTC: Real-time robot execution with anticipatory-conditioned action chunking,” *arXiv preprint arXiv:2607.24008*, 2026.
- [15] Z. Wang, X. Zhou, C. Xu, and F. Gao, “Geometrically constrained trajectory optimization for multicopters,” *IEEE Transactions on Robotics*, vol. 38, no. 5, pp. 3259–3278, 2022.
- [16] W. Deng, H. Chen, B. Ye, H. Chen, Z. Li, and X. Lyu, “Whole-body integrated motion planning for aerial manipulators,” *IEEE Transactions on Robotics*, vol. 41, pp. 6661–6679, 2025.
- [17] Q. Gu, A. Kuwajerwala, S. Morin, K. M. Jatavallabhula, B. Sen, A. Agarwal, C. Rivera, W. Paul, K. Ellis, R. Chellappa, C. Gan, C. M. de Melo, J. B. Tenenbaum, A. Torralba, F. Shkurti, and L. Paull, “ConceptGraphs: Open-vocabulary 3D scene graphs for perception and planning,” *arXiv preprint arXiv:2309.16650*, 2024.
- [18] A. Werby, C. Huang, M. Büchner, A. Valada, and W. Burgard, “Hierarchical open-vocabulary 3D scene graphs for language-grounded robot navigation,” in *Robotics: Science and Systems*, 2024.
- [19] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Sünderhauf, “SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning,” in *Conference on Robot Learning*, 2023.
- [20] Z. Yan, S. Li, Z. Wang, L. Wu, H. Wang, J. Zhu, L. Chen, and J. Liu, “Dynamic open-vocabulary 3D scene graphs for long-term language-guided mobile manipulation,” *IEEE Robotics and Automation Letters*, 2025.
- [21] W. Gai, Y. Gao, Y. Zhou, Y. Xie, Z. Liu, Y. Wu, X. Zhou, F. Gao, and Z. Meng, “USS-Nav: Unified spatio-semantic scene graph for lightweight UAV zero-shot object navigation,” *arXiv preprint arXiv:2602.00708*, 2026.
- [22] Physical Intelligence *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.